Лабораторная работа №4
«АРХИТЕКТУРА СИСТЕМ»

4.1 Цель работы: Изучить основы моделирования архитектуры системы, а также построения платформо-зависимой модели. Выполнить индивидуальное задание в среде Enterprise Architect.

Задание: спроектировать структуру информационной системы в соответствии с выбранной предметной областью в виде диаграммы компонентов и диаграммы развертывания.

4.2 Теоретические сведения

Компоненты – это физические части системы. Следовательно, проектирование компонент нельзя отделить от платформы реализации. Проектируемая система онлайновой торговли – это Web-приложение с сервером баз данных. Web-приложение – это Web-система, позволяющая ее пользователям работать в соответствии с заложенной в ней бизнес - логикой с помощью Web-браузера. Программа, поддерживающая бизнес-логику, может находить на сервере и/или на клиенте. Следовательно, Web-приложение – не что иное, как разновидность системы клиент-сервер с Web-узлом. Относительно нашего примера, рассмотрим типичную последовательность доступа к Web-страницам. Первая Web-страница, которую может посетить пользователь, – это Web-страница поставщика, на которой перечислены группы изделий (серверные, настольные системы, портативные компьютеры) и приводятся ссылки на Web-страницы, на которых представлены перечни изделий и дано краткое описание каждого изделия. Это единый функциональный блок, который может образовать компонент ProductList (Перечень изделий) и т.д. В соответствии с этим построим диаграмму компонентов. Важно отметить сходство этой

диаграммы с диаграммой пакетов прецедентов, что неудивительно, поскольку пакеты прецедентов и компоненты представляют собой функциональные модули с четким границами.

Итак, на диаграмме компонентов отражаются программные компоненты системы. Для того, чтобы создать ее в Enterprise Architect, выберите группу *UML Structural*, затем *Component*.

Характер Internet-систем без установления прямого соединения делает развертывание Web-приложений значительно более сложной задачей, чем развертывание приложений баз данных в архитектуре клиент-сервер. Чтобы приступить к развертыванию, требуется установить Web-сервер в качестве пункта маршрутизации между всеми браузерами клиентов и базой данных. На рис. 20 показана диаграмма развертывания для нашего примера. Система онлайновой торговли развернута без отдельного сервера приложений.

Таким образом, диаграмма развертывания отражает систему в ее работе с программными и аппаратными ресурсами.

Для того чтобы создать ее в Enterprise Architect, выберите группу *UML Structural*, затем *Deployment*.

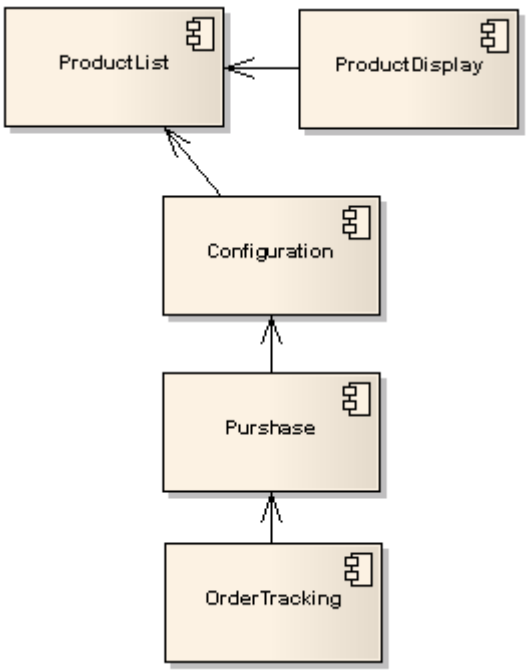


Рис. 19. Диаграмма компонентов

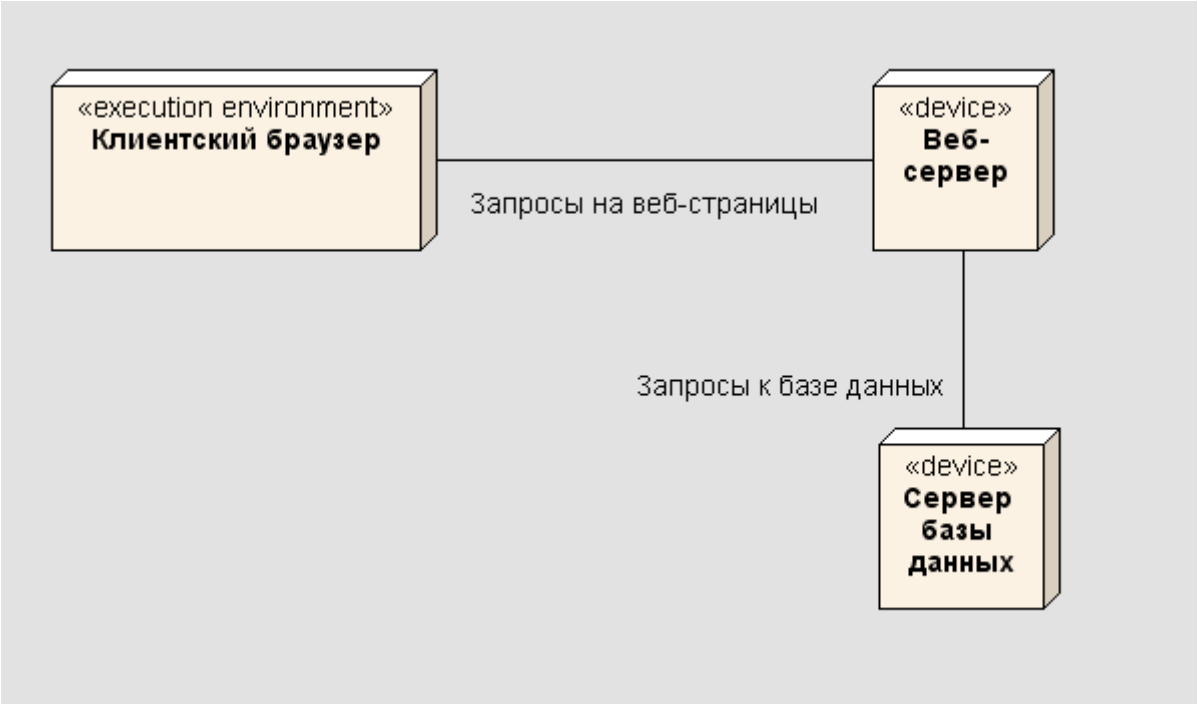


Рис. 20. Пример диаграммы развертывания.

Модельно-ориентированная архитектура (Model Driven Architecture, далее – MOA) представляет собой стратегическую инициативу консорциума Object Management Group (OMG), пришедшую в конце 2000 г. на смену архитектуре управления объектами (Object Management Architecture), которая служила фундаментом платформы распределенных объектных вычислений CORBA.

Основным отличием модельно-ориентированного подхода от прочих (структурного, объектно-ориентированного) является выбор в качестве конечной цели разработки не конкретной реализации системы, а некоторой модели системы или совокупности таких моделей (например, модели организации согласно архитектуре Захмана или иной аналогичной методологии). При этом происходит перераспределение ресурсов времени и персонала: уменьшение значимости стадии кодирования и прирост стадии анализа и проектирования.

Организация OMG уже приняла в качестве дополнения к стандарту UML ряд профилей, предназначенных, в том числе, для спецификации вычислений в распределенных объектных средах (UML Profile for EDOC), интеграции приложений уровня предприятия (UML Profile for EAI) и т.п.

Профили, как и UML-модели, можно разделить на платформо-зависимые (например, профиль для CORBA-приложений) и платформо-независимые (EDOC, EAI).

Критерий платформо-зависимости является в рамках MOA ключевым для построения иерархии моделей системы. В пределах каждого уровня (бизнес-модели, платформо-независимые, платформо-зависимые модели) может существовать множество моделей, отличающихся по степени абстракции, по точке зрения на систему и т.п. Между моделями соседних уровней иерархии MOA устанавливаются отношения уточнения (refinement), в которых более платформо-зависимые модели уточняют менее платформо-зависимые.

Подобная иерархия моделей обладает следующими достоинствами:

1) для каждого заинтересованного лица строится модель, соответствующая его уровню понимания и точке зрения на систему (для члена совета директоров – бизнес-модель, для начальника отдела АСУ – платформо-независимая модель, для рядового программиста – платформо-зависимая, возможно даже в виде каркаса исходного кода);

2) обеспечивается естественная поддержка множества распространенных методологий разработки ПО. Возможность взаимообратного преобразования PIM и PSM моделей является одной из ключевых особенностей MOA. Преобразование моделей в рамках MOA используется для следующих целей:

2.1. преобразование и уточнение UML-моделей (трансформация PIM в PSM модели);

2.2. исследование и оценка UML-моделей при помощи других методик моделирования (например, построение по UML-модели динамической модели для исследования поведения проектируемой системы);

2.3. генерация исходного кода.

Для того, чтобы по готовой к реализации диаграмме классов сгенерировать каркасный код программных компонентов, откройте вашу диаграмму классов, в меню *Project* выберите пункт *Source Code Engineering*, затем *Generate Package Source Code*. Или – на панели *Project Browser* выберите список *Code Generation*, затем – *Generate Source Code*.

В появившемся окне появится список компонентов диаграммы классов, выделите те из них, для которых хотите получить каркасный код, затем можете изменить некоторые опции, если вам нужно, и нажать кнопку *Generate*.

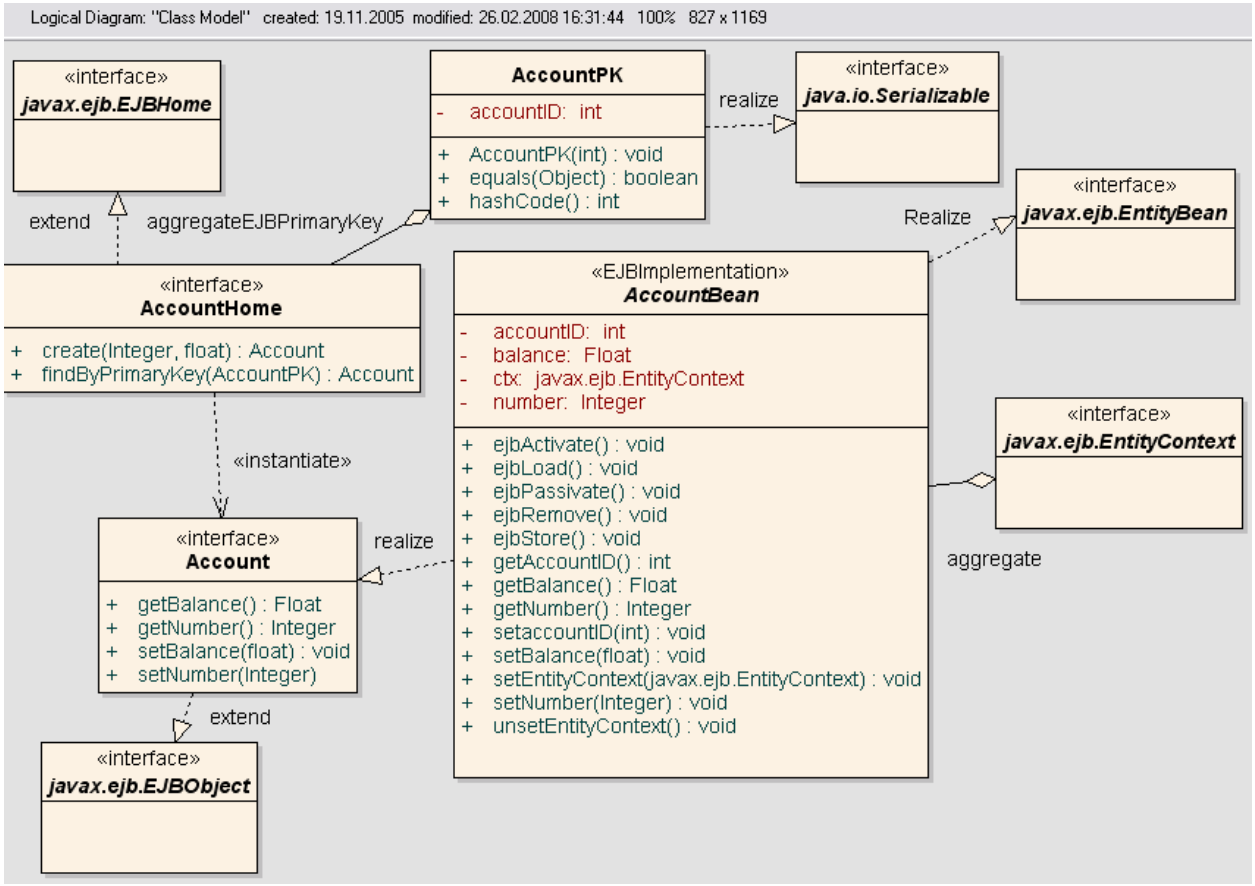


Рис. 21. Пример построения платформо - зависимой модели.

Программа будет для каждого компонента спрашивать, в какую папку его положить и как должен называться файл.

Вот исходный каркасный код для EJB-компонента AccountBean из нашего примера:

```
/**
```

```
 * @version 1.0
 * @created 26-фев-2008 17:04:14
 */
public abstract class AccountBean extends Account implements javax.ejb.EntityBean, EJBEntityHomeInterface {
	private int accountID;
	private Float balance;
	private javax.ejb.EntityContext ctx;
	private Integer number;
	public AccountBean(){
	}
	public void finalize() throws Throwable {
	}
	public void ejbActivate(){
	}
	public void ejbLoad(){
	}
	public void ejbPassivate(){
	}
	public void ejbRemove(){
	}
	public void ejbStore(){
	}
	public int getAccountID(){
		return 0;
	}
	public Float getBalance(){
		return null;
	}
	public Integer getNumber(){
		return 0;
	}
	/**
	 *
	 * @param accountID
	 */
	public void setaccountID(int accountID){
	}
	/**
	 *
	 * @param balance
	 */
```

```
    public void setBalance(float balance){
    }
    /**
     *
     * @param entityContext
     */
    public void setEntityContext(javax.ejb.EntityContext entityContext){
    }
    /**
     *
     * @param number
     */
    public void setNumber(Integer number){
    }
    public void unsetEntityContext(){
    }
}
```

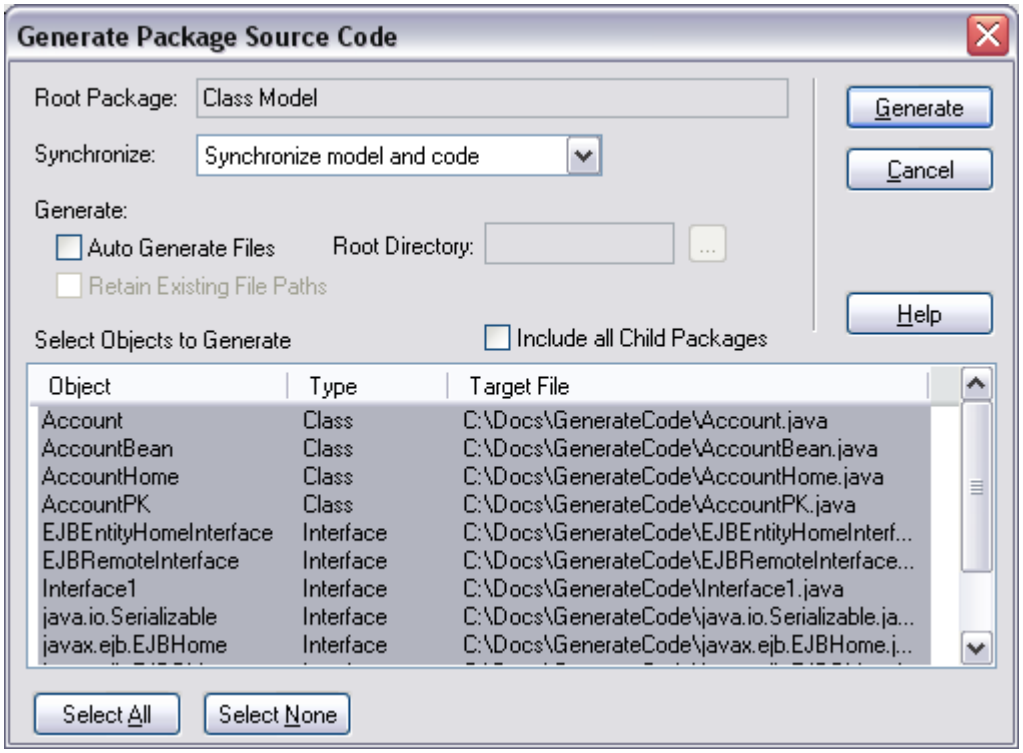


Рис. 22. Подготовка к генерации кода.

4.3 Порядок выполнения.

1. Получите у преподавателя вариант индивидуального задания, или предложите свой, который должен содержать диаграммы компонентов,

развертывания и диаграмму классов платформо - зависимой модели, и согласуйте его с преподавателем.

2. Выполните в Enterprise Architect ваш вариант задания.

3. Составить отчет о работе.

4.4 Содержание отчета.

1. Цель работы.

2. Описание предметной области.

3. Диаграммы Enterprise Architect с пояснениями.

4. Сгенерированный каркасный код программных компонентов.

5.Вывод.

4.5 Контрольные вопросы.

1. Поясните сущность диаграмм компонентов.
2. Поясните сущность диаграмм развертывания.
3. Поясните сущность модельно-ориентированной архитектуры.
4. Как вы считаете, всегда ли более актуальными являются именно платформо – зависимые модели?